

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ

УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

(МОСКОВСКИЙ ПОЛИТЕХ)

КУРСОВОЙ ПРОЕКТ

по курсу

«Разработка веб-приложений»

ТЕМА

**«Разработка веб-приложения для автоматизации процесса
бронирования столиков в ресторане»**

Выполнила: Антипова Анастасия Максимовна

Группа: 241-3211

Проверил: Кружалов Алексей Сергеевич

Москва, 2026

**«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(МОСКОВСКИЙ ПОЛИТЕХ)**

УТВЕРЖДАЮ
заведующая кафедрой
«Инфокогнитивные технологии»

_____ / Е. А. Пухова /

«_____» _____ 2026 г.

ЗАДАНИЕ

на выполнение курсовой работы (проекта)

Антиповой Анастасии Максимовне,

обучающейся группы 241-3211,

направления подготовки 09.03.01 «Информатика и вычислительная техника»

по дисциплине «Разработка веб-приложений»

на тему «Разработка веб-приложения для автоматизации процесса бронирования столиков в ресторане»

1. Исходные данные к работе (проекту): информационные ресурсы в сети интернет, научные публикации в открытой печати.

2. Содержание задания по курсовой работе (проекту) – перечень вопросов, подлежащих разработке:

Разрабатываемый вопрос	Объем, %	Срок	Примечание
Раздел 1. Анализ предметной области			
Задача 1.1. Обзор существующих программных продуктов по теме работы	10	10.04.2026	
Задача 1.2. Анализ программных инструментов разработки веб-приложений	7	13.04.2026	
Задача 1.3. Формулировка цели и задач работы	8	18.04.2026	
Раздел 2. Проектирование веб-приложения			
Задача 2.1. Анализ целевой аудитории	5	25.04.2026	
Задача 2.2. Описание функциональности приложения (диаграмма вариантов использования, user story)	7	30.04.2026	

Разрабатываемый вопрос	Объем, %	Срок	Примечание
Задача 2.3. Проектирование модели данных (ER-диаграмма, логическая и физическая схемы БД)	8	05.05.2026	
Задача 2.4. Разработка макетов страниц (Wireframe)	5	16.05.2026	
Раздел 3. Разработка веб-приложения			
Задача 3.1. Разработка базовой структуры приложения и вёрстка шаблонов страниц	8	18.05.2026	
Задача 3.2. Реализация аутентификации пользователей	7	25.05.2026	
Задача 3.3. Реализация CRUD-интерфейса для управления столиками и бронированиями, включая загрузку изображений столиков и меню	7	29.05.2026	
Задача 3.4. Реализация системы проверки доступности столиков и управления временем бронирования с поддержкой экспорта отчётов о занятости столиков (CSV, PDF)	12	02.06.2026	
Задача 3.5. Разработка системы уведомлений и генерации подтверждений бронирования с вложением файлов (PDF-чеки)	6	06.06.2026	
Раздел 4. Оформление итогов работы			
Задача 4.1. Создание Git-репозитория с кодом проекта	3	10.04.2026	
Задача 4.2. Деплой приложения на хостинг	4	10.06.2026	
Задача 4.3. Оформление отчёта о проделанной работе	3	13.06.2026	

Руководитель курсовой работы (проекта):
старший преподаватель кафедры «Инфокогнитивные технологии»

«____» _____ 2026 г.

(подпись)

А. С. Кружалов

Дата выдачи задания «2» апреля 2026 г.

Дата сдачи выполненной работы «____» _____ 2026 г.

Задание принял к исполнению

«2» апреля 2026 г.



(подпись)

А. М. Антипова

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	5
Глава 1. Анализ предметной области	7
1.1 Обзор существующих программных продуктов по теме работы .	7
1.2 Анализ программных инструментов разработки веб-приложений	8
Глава 2. Проектирование веб-приложения	10
2.1 Анализ целевой аудитории	10
2.2 Описание функциональности приложения	11
2.3 Проектирование модели данных	13
2.4 Разработка шаблонов страниц	17
Пользовательские шаблоны	17
Административные шаблоны	20
3 Реализация веб-приложения	23
3.1 Выбор архитектуры и структуры проекта	23
3.2 Реализация серверной части	24
3.3 Реализация базы данных	25
3.4 Реализация аутентификации пользователей	26
3.5 Реализация бронирования и проверки доступности столиков . .	26
3.6 Реализация административной панели	27
3.7 Реализация уведомлений, PDF-подтверждений и email-рассылки	28
3.8 Реализация клиентской части	28
3.9 Запуск и тестирование приложения	29
3.10 Выводы по разделу	31
ЗАКЛЮЧЕНИЕ	32
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	33

ВВЕДЕНИЕ

Актуальность темы. В условиях активной цифровизации сферы услуг автоматизация ключевых бизнес-процессов ресторанного бизнеса становится важным условием повышения качества обслуживания. Процесс бронирования столиков остаётся одним из наиболее трудоёмких в части ручного учёта: администраторы вынуждены вести резервации в журналах или электронных таблицах, что может приводить к ошибкам, двойным бронированиям и потере клиентов. Разработка специализированного веб-приложения, автоматизирующего цикл резервации столиков, является актуальной практической задачей.

Степень разработанности проблемы. На рынке существуют платформы бронирования и автоматизации ресторанного бизнеса, такие как Restoplace, Yclients и iiko [1—3]. Однако подобные решения могут быть избыточными для небольшого заведения, которому требуется простая и адаптируемая система управления бронированием. Поэтому разработка гибкого веб-приложения, легко адаптируемого под нужды конкретного ресторана, остаётся актуальной задачей.

Объект исследования — процесс управления бронированием столиков в ресторане.

Предмет исследования — программные средства и технологии разработки веб-приложений для автоматизации процесса резервации столиков, уведомления авторизованных пользователей и формирования отчётности.

Цель работы — разработка веб-приложения для автоматизации процесса бронирования столиков в ресторане, обеспечивающего управление резервациями, уведомление авторизованных пользователей и формирование отчётности.

Для достижения поставленной цели необходимо решить следующие задачи:

1. провести обзор существующих систем бронирования и анализ инструментов разработки с целью обоснования выбора технологического стека;
2. проанализировать целевую аудиторию и описать функциональные требования к приложению посредством диаграммы вариантов использования и пользовательских историй (User Story);
3. спроектировать модель данных: разработать диаграмму сущность-связь

- (ER-диаграмму), логическую и физическую схемы базы данных;
4. разработать макеты страниц (Wireframe) и реализовать базовую структуру приложения с вёрсткой шаблонов страниц;
 5. реализовать систему аутентификации пользователей и интерфейс управления данными (CRUD) для столиков и бронирований, включая загрузку изображений;
 6. реализовать систему проверки доступности столиков и управления временем бронирования с поддержкой экспорта отчётов о занятости (CSV, PDF);
 7. разработать систему уведомлений и генерации PDF-подтверждений бронирования с отправкой авторизованным пользователям по электронной почте;
 8. создать репозиторий с исходным кодом проекта и выполнить развёртывание приложения на хостинге.

Методы исследования: анализ и сравнение программных продуктов, проектирование архитектуры веб-приложений, объектно-ориентированное программирование, прототипирование интерфейсов.

Практическая значимость работы состоит в том, что разработанное приложение может быть непосредственно внедрено в работу ресторана и заменить ручной учёт резервации столиков, сократив количество ошибок и повысив качество обслуживания пользователей.

Глава 1. Анализ предметной области

1.1 Обзор существующих программных продуктов по теме работы

На современном рынке представлен ряд программных решений для автоматизации бронирования столиков в ресторанах. Для объективной оценки существующих систем были определены критерии, которым должно удовлетворять разрабатываемое приложение: онлайн-бронирование авторизованным пользователем без звонка в заведение; интерактивная схема зала с отображением статуса столиков; автоматическая проверка пересечений бронирований; отправка email-уведомлений с PDF-подтверждением; экспорт отчётов о занятости в форматах CSV и PDF; отсутствие значительных постоянных расходов; возможность адаптации логики под конкретное заведение.

В соответствии с перечисленными критериями рассмотрены три системы: **iiko**, **Restoplace** и **Yclients** [1—3].

Система **iiko** — российская платформа комплексной автоматизации ресторана (касса, кухня, склад, персонал). Управление схемой зала доступно администратору, однако онлайн-бронирование авторизованным пользователем не предусмотрено: резервации вносятся вручную сотрудником. Стоимость внедрения составляет от 50 000 рублей, что нецелесообразно для заведений, которым требуется только онлайн-бронирование.

Сервис **Restoplace** — отечественная облачная платформа, специализирующаяся на бронировании столиков. Авторизованный пользователь самостоятельно выбирает столик на интерактивной схеме зала через виджет на сайте или во ВКонтакте. Бесплатный тариф ограничен 300 заявками в месяц. Принципиальным ограничением является отсутствие генерации PDF-подтверждений и экспорта отчётов — функций, требуемых в данном проекте.

Сервис **Yclients** — наиболее распространённая в России CRM-система для сферы услуг (более 26 000 подключённых заведений). Предоставляет виджет онлайн-записи, SMS/email-уведомления, интеграцию с кассами (АТОЛ, Эвотор). Однако система изначально ориентирована на запись к мастеру, а не на ресторанное бронирование: нативная схема зала отсутствует, подписка стоит от 3 000 рублей в месяц.

Результаты сравнения систем по выделенным критериям приведены в таблице 1.

Анализ показал, что ни одна из рассмотренных систем не удовлетворя-

Таблица 1 – Сравнительный анализ существующих систем по критериям

Критерий	iiko	Restoplace	Yclients	Собственная разработка
Онлайн-бронирование авторизованным пользователем	—*	+	+	+
Интерактивная схема зала	+	+	—	+
Проверка доступности столиков	+	+	+	+
Email-уведомления и PDF-подтверждения	—	—	—	+
Экспорт отчётов (CSV, PDF)	—	—	—	+
Доступность для малого бизнеса	—	+ / —	+ / —	+
Возможность кастомизации	—	—	—	+

* бронирование вносится вручную сотрудником; самостоятельная онлайн-резервация авторизованным пользователем не предусмотрена.

ет всем критериям одновременно. Ключевыми незакрытыми потребностями являются генерация PDF-подтверждений и экспорт отчётов о занятости. Разработка собственного веб-приложения, полностью адаптированного к требованиям заведения, является обоснованной и целесообразной.

1.2 Анализ программных инструментов разработки веб-приложений

Перед выбором технологического стека проведён обзор инструментов, актуальных для современной fullstack-разработки, по четырём уровням.

Клиентская часть. Для построения интерфейсов веб-приложений широко применяются React, Vue.js и Angular [4]. React — библиотека, основанная на компонентном подходе, с наибольшей экосистемой. Vue.js прост в освоении, но уступает по объёму готовых библиотек. Angular ориентирован на крупные корпоративные проекты и требует значительного времени на конфигурацию. Для проекта выбран **React**: предоставляет готовые компоненты для интерактивных форм бронирования (react-calendar, react-toastify).

Серверная часть. Среди актуальных решений выделяются Node.js, Django, FastAPI и Spring Boot [5]. Node.js эффективен при большом числе

соединений, но не имеет встроенных аутентификации и ORM. FastAPI обеспечивает быструю разработку API, однако лишён административного интерфейса. Spring Boot масштабируем, но требует длительной конфигурации. Для проекта выбран **Django**: предоставляет встроенную аутентификацию, ORM и административную панель, что напрямую закрывает требования задания.

База данных. Наиболее распространены PostgreSQL, MySQL и SQLite [6]. SQLite не поддерживает параллельную запись и непригодна для production. MySQL уступает PostgreSQL по строгости транзакций. Для проекта выбрана **PostgreSQL**: обеспечивает надёжную обработку конкурентных запросов при одновременном бронировании одного столика, полностью совместима с Django ORM.

Среда развёртывания. Традиционная ручная настройка сервера создаёт риск расхождения окружений. Контейнеризация с помощью **Docker** и docker-compose устраняет эту проблему: все сервисы описываются в едином файле и воспроизводятся на любом хосте [7].

Глава 2. Проектирование веб-приложения

2.1 Анализ целевой аудитории

Разрабатываемое приложение RestaurantBook предназначено для автоматизации процесса бронирования столиков в ресторане. Система ориентирована на две основные группы пользователей: авторизованных пользователей и администраторов. Такое разделение необходимо, поскольку пользовательские сценарии у этих ролей различаются: авторизованный пользователь создаёт и отслеживает бронирование, а администратор управляет данными, контролирует занятость столиков и формирует отчёты.

Авторизованный пользователь — посетитель ресторана, который после регистрации и входа в систему может заранее выбрать дату, время, столик и получить подтверждение без телефонного звонка. Для данной роли важны простая регистрация, просмотр меню, выбор свободного столика, получение уведомлений в интерфейсе, email-уведомлений и PDF-подтверждения бронирования.

Администратор — сотрудник ресторана, который отвечает за актуальность информации в системе. Он управляет столиками, меню, бронированиями, статусами заявок, изображениями столиков и блюд, а также получает отчёты о загрузженности зала в форматах CSV и PDF.

Для описания целевой аудитории применён метод персонажей (Personas), позволяющий формализовать потребности и типовые сценарии поведения каждой роли [8]. Разграничение прав доступа реализуется на уровне аутентификации: обычный пользователь получает доступ к пользовательским страницам, а администратор — к панели управления.

Таблица 2 – Характеристика целевой аудитории приложения

Роль	Основные потребности	Ключевые функции системы
Авторизованный пользователь	Забронировать столик, посмотреть меню, получить подтверждение и уведомления	Регистрация, вход, просмотр меню, проверка доступности столика, бронирование, внутренние и email-уведомления, PDF-подтверждение
Администратор	Управлять столиками, меню и бронированиями, контролировать загрузку зала	CRUD-интерфейс, загрузка изображений, изменение времени и статусов бронирований, отчёты CSV/PDF

2.2 Описание функциональности приложения

Функциональность системы была описана через пользовательские истории (User Story). Такой формат позволяет зафиксировать требование с точки зрения конкретной роли: кто выполняет действие, какую возможность он получает и зачем она нужна [9]. В таблице 3 пользовательские истории сформулированы так, чтобы показать основные сценарии работы пользователя и администратора с системой бронирования.

Таблица 3 – Пользовательские истории системы RestaurantBook

Роль	User Story	Результат
Авторизованный пользователь	Я, как пользователь, хочу зарегистрироваться и войти в систему	Доступ к личным бронированиям и уведомлениям
Авторизованный пользователь	Я, как пользователь, хочу посмотреть меню ресторана	Возможность выбрать ресторанное предложение до бронирования

Роль	User Story	Результат
Авторизованный пользователь	Я, как пользователь, хочу проверить доступность столика на выбранные дату и время	Исключение выбора занятого столика
Авторизованный пользователь	Я, как пользователь, хочу создать бронирование с указанием времени начала и окончания	Фиксация резервации в системе
Авторизованный пользователь	Я, как пользователь, хочу получить уведомление в интерфейсе, email-сообщение и PDF-подтверждение после бронирования	Подтверждение факта бронирования и информирование пользователя
Администратор	Я, как администратор, хочу выполнять CRUD-операции со столиками	Актуальное состояние схемы зала
Администратор	Я, как администратор, хочу выполнять CRUD-операции с бронированиями	Возможность подтвердить, изменить или отменить заявку
Администратор	Я, как администратор, хочу загружать изображения столиков и блюд меню	Наглядное отображение данных в интерфейсе
Администратор	Я, как администратор, хочу экспортировать отчёты о занятости в CSV и PDF	Получение отчётности по загрузке столиков

На рисунке 1 представлена диаграмма вариантов использования системы, построенная в нотации UML [10]. В диаграмме отдельно показаны регистрация и вход, CRUD столиков и меню, CRUD бронирований, загрузка изображений, проверка доступности столика, управление временем бронирования, экспорт отчётов CSV/PDF, просмотр уведомлений, получение email-уведомлений и генерация PDF-подтверждения.



Рисунок 1 – Диаграмма вариантов использования системы

2.3 Проектирование модели данных

Для хранения данных приложения спроектирована реляционная модель. В неё включены сущности пользователей, столиков, позиций меню, бронирований, уведомлений и файлов подтверждений. Метод диаграмм сущность-связь позволяет наглядно описать структуру данных и отношения между сущностями [11]. На рисунке 2 представлена ER-диаграмма. Связь между пользователем и бронированием вынесена в обход центральной сущности, поэтому линия не проходит сквозь таблицы и не ухудшает читаемость схемы.

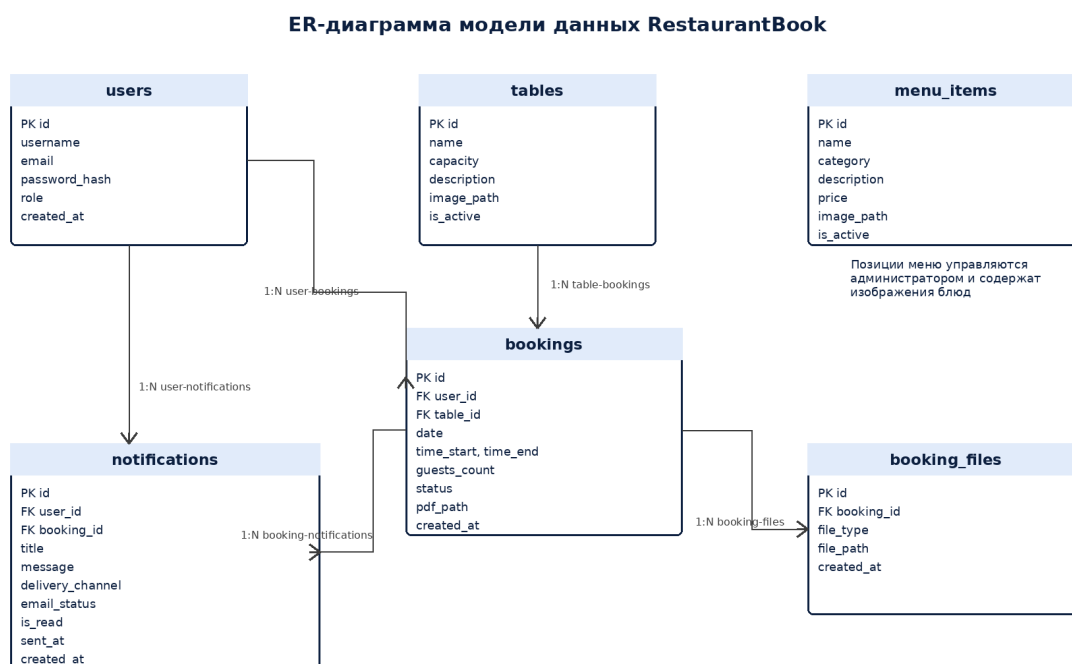


Рисунок 2 – ER-диаграмма модели данных

Сущность `users` используется для хранения данных пользователей: логина, `email`, хэша пароля и роли. Сущности `tables`, `menu_items` и `bookings` обеспечивают работу со столиками, меню и бронированиями. Поля `image_path` используются для хранения путей к загруженным изображениям столиков и блюд меню. Сущность `notifications` используется для хранения внутренних уведомлений и сведений об email-рассылке. Поле `delivery_channel` показывает канал доставки уведомления: внутри сайта или на электронную почту. Поля `email_status` и `sent_at` позволяют фиксировать результат и время отправки письма. Сущность `booking_files` используется для файлов подтверждений бронирования. Она хранит сведения о сформированном PDF-чеке: тип файла, путь к файлу и дату генерации. Благодаря отдельной таблице можно хранить историю подтверждений и связывать каждый файл с конкретным бронированием.

Структура таблицы пользователей приведена в таблице 4.

Таблица 4 – Структура таблицы пользователей (`users`)

Поле	Тип	Ограничение	Описание
id	INTEGER	PRIMARY KEY, AUTO	Уникальный идентификатор пользователя

Поле	Тип	Ограничение	Описание
username	VARCHAR	NOT NULL, UNIQUE	Логин пользователя
email	VARCHAR	NOT NULL, UNIQUE	Адрес электронной почты
password_hash	VARCHAR	NOT NULL	Хэш пароля пользователя
role	VARCHAR	DEFAULT 'user'	Роль: пользователь или администратор
created_at	TIMESTAMP	DEFAULT NOW()	Дата и время регистрации

Структура таблицы столиков приведена в таблице 5.

Таблица 5 – Структура таблицы столиков (tables)

Поле	Тип	Ограничение	Описание
id	INTEGER	PRIMARY KEY, AUTO	Уникальный идентификатор столика
name	VARCHAR	NOT NULL	Название или номер столика
capacity	INTEGER	NOT NULL	Максимальное количество гостей
description	TEXT	NULL	Описание расположения и особенностей
image_path	VARCHAR	NULL	Путь к фотографии столика
is_active	BOOLEAN	DEFAULT TRUE	Доступность столика для бронирования

Структура таблицы меню приведена в таблице 6.

Таблица 6 – Структура таблицы меню (menu_items)

Поле	Тип	Ограничение	Описание
id	INTEGER	PRIMARY KEY, AUTO	Уникальный идентификатор блюда
name	VARCHAR	NOT NULL	Название блюда
category	VARCHAR	NOT NULL	Категория меню
description	TEXT	NULL	Описание блюда
price	DECIMAL	NOT NULL	Стоимость позиции
image_path	VARCHAR	NULL	Путь к изображению блюда
is_active	BOOLEAN	DEFAULT TRUE	Отображение позиции в меню

Структура таблицы бронирований приведена в таблице 7.

Таблица 7 – Структура таблицы бронирований (bookings)

Поле	Тип	Ограничение	Описание
id	INTEGER	PRIMARY KEY, AUTO	Уникальный идентификатор бронирования
user_id	INTEGER	FOREIGN KEY	Ссылка на пользователя
table_id	INTEGER	FOREIGN KEY	Ссылка на столик
date	DATE	NOT NULL	Дата бронирования
time_start	TIME	NOT NULL	Время начала резервации
time_end	TIME	NOT NULL	Время окончания резервации
guests_count	INTEGER	NOT NULL	Количество гостей
status	VARCHAR	DEFAULT 'pending'	Статус бронирования
pdf_path	VARCHAR	NULL	Путь к PDF-чеку бронирования
created_at	TIMESTAMP	DEFAULT NOW()	Дата и время создания записи

Структура таблицы уведомлений приведена в таблице 8.

Таблица 8 – Структура таблицы уведомлений (notifications)

Поле	Тип	Ограничение	Описание
id	INTEGER	PRIMARY KEY, AUTO	Уникальный идентификатор уведомления
user_id	INTEGER	FOREIGN KEY	Получатель уведомления
booking_id	INTEGER	FOREIGN KEY, NULL	Связанное бронирование
title	VARCHAR	NOT NULL	Заголовок уведомления
message	TEXT	NOT NULL	Текст уведомления
delivery_channel	VARCHAR	DEFAULT 'site'	Канал доставки: интерфейс сайта или email
email_status	VARCHAR	NULL	Статус отправки письма
is_read	BOOLEAN	DEFAULT FALSE	Признак прочтения в интерфейсе
sent_at	TIMESTAMP	NULL	Дата и время отправки email
created_at	TIMESTAMP	DEFAULT NOW()	Дата создания уведомления

Структура таблицы файлов подтверждений приведена в таблице 9.

Таблица 9 – Структура таблицы файлов подтверждений (booking_files)

Поле	Тип	Ограничение	Описание
id	INTEGER	PRIMARY KEY, AUTO	Уникальный идентификатор файла

Поле	Тип	Ограничение	Описание
booking_id	INTEGER	FOREIGN KEY	Связанное бронирование
file_type	VARCHAR	DEFAULT 'pdf'	Тип сформированного файла
file_path	VARCHAR	NOT NULL	Путь к PDF-чеку бронирования
generated_at	TIMESTAMP	DEFAULT NOW()	Дата и время генерации файла

Алгоритм проверки доступности столика основан на сравнении временных интервалов. При создании новой заявки система получает выбранный столик, дату, время начала и окончания бронирования, затем ищет существующие записи с тем же столиком и датой. Если найдено активное бронирование, у которого интервалы пересекаются, пользователю выводится сообщение о недоступности выбранного времени. Если пересечений нет, запись создаётся, формируется PDF-подтверждение, а пользователю отправляется уведомление в интерфейсе и email-сообщение на адрес, указанный при регистрации. Email содержит дату, время, выбранный столик, количество гостей и при необходимости вложенный PDF-файл подтверждения. Такой способ информирования не требует реализации push-уведомлений и позволяет уведомить пользователя даже тогда, когда он не находится на сайте.

2.4 Разработка шаблонов страниц

Для проверки структуры интерфейса были подготовлены шаблоны ключевых страниц приложения. Шаблоны разделены на пользовательские и административные, так как эти группы экранов соответствуют разным ролям и правам доступа. На пользовательских страницах в верхней панели предусмотрен значок колокольчика. Если есть новые уведомления, рядом с иконкой отображается числовой индикатор, а нажатие на неё переводит пользователя к списку уведомлений.

Пользовательские шаблоны

На рисунке 3 представлен шаблон страниц входа и регистрации. Пользователь вводит логин, email и пароль, а система отображает ошибку при неверных данных и позволяет перейти между формами входа и регистрации.

RestaurantBook

Меню Бронирование Увед. 2 Выйти

Аутентификация

Вход и регистрация пользователя в системе

Вход в систему

Логин

Пароль

Войти

Данные отправляются на Django API

Регистрация

Логин

Email

Пароль

Создать аккаунт

Данные отправляются на Django API

Рисунок 3 – Пользовательский шаблон страниц входа и регистрации

На рисунке 4 представлен шаблон страницы меню ресторана. На странице показаны категории блюд, карточки с названием, описанием, ценой и изображением. Изображения блюд загружаются через административный CRUD-интерфейс и отображаются пользователю в меню.

RestaurantBook

Меню Бронирование Увед. 2 Выйти

Меню ресторана

Все блюда Салаты Горячее Десерты Напитки



Цезарь
Салаты

590 ₽



Паста
Горячее

780 ₽



Стейк
Горячее

1450 ₽



Чизкейк
Десерты

420 ₽



Лимонад
Напитки

300 ₽



Брускетта
Закуски

490 ₽

Рисунок 4 – Пользовательский шаблон страницы меню ресторана

На рисунке 5 представлен шаблон страницы бронирования. Пользователь выбирает дату, время начала и окончания, количество гостей и столик. На основе этих данных система проверяет доступность столика и создаёт бронирование только при отсутствии пересечений по времени. В шаблоне

также предусмотрено информационное сообщение о доступности выбранного столика для указанного интервала, а после создания заявки пользователь получает внутреннее уведомление и email-подтверждение.

Рисунок 5 – Пользовательский шаблон страницы бронирования столика

На рисунке 6 представлен шаблон страницы уведомлений. Пользователь видит уведомления о создании, подтверждении, напоминании и отмене бронирования. В тексте уведомления предусмотрено сообщение о PDF-подтверждении, отправленном на почту. Email-рассылка дублирует наиболее важные уведомления: подтверждение бронирования, напоминание, отмену или изменение статуса.

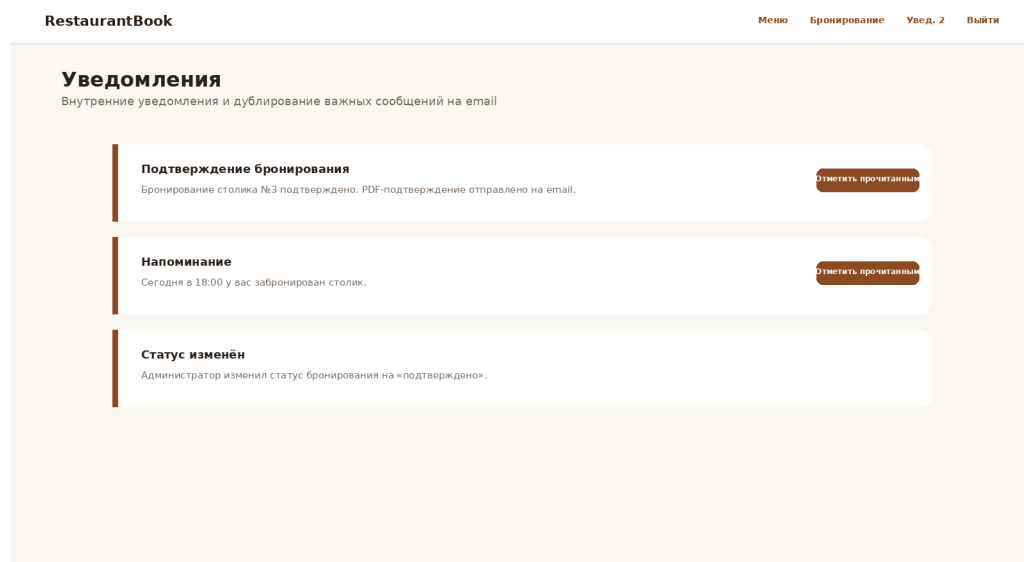


Рисунок 6 – Пользовательский шаблон страницы уведомлений

Административные шаблоны

На рисунке 7 представлен шаблон административной панели бронирований. Администратор фильтрует заявки по дате, столику и статусу, видит время бронирования, изменяет или удаляет записи, а также экспортирует данные в CSV и PDF.

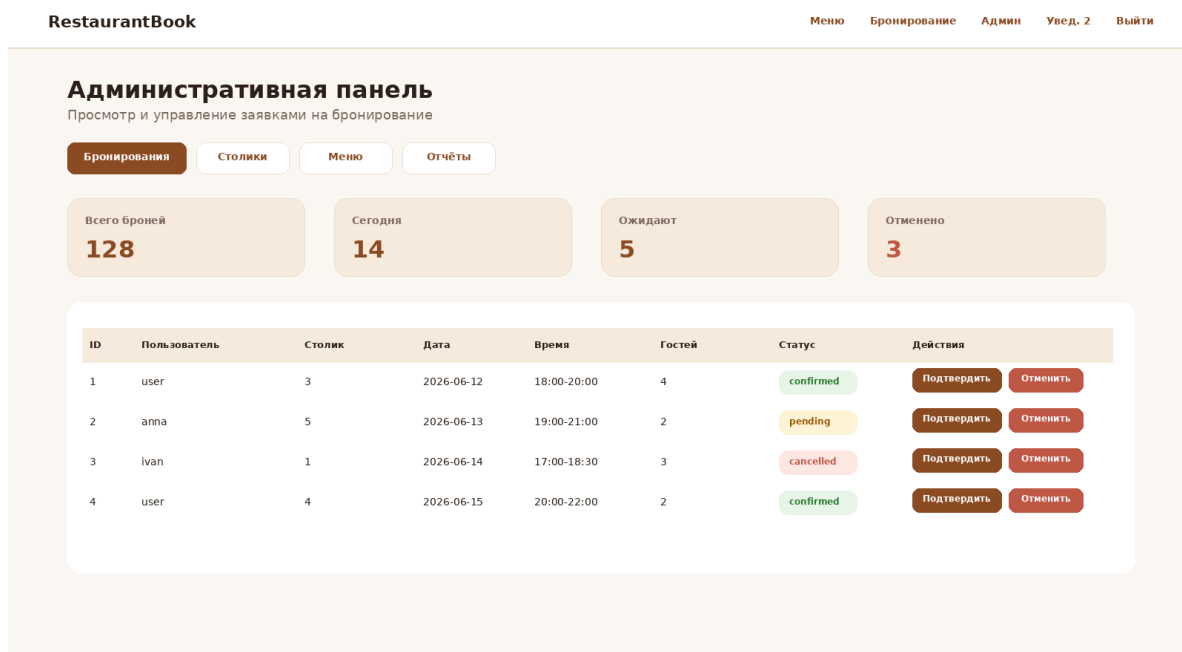


Рисунок 7 – Административный шаблон управления бронированиями

На рисунке 8 представлен шаблон административного раздела управления столиками и меню. В интерфейсе предусмотрены добавление, редактирование и удаление записей, переключение между вкладками «Столики» и «Меню», а также загрузка фотографии столика и изображения позиции меню.

RestaurantBook Меню Бронирование Админ Увед. 2 Выйти

Административная панель

Добавление и редактирование столиков и позиций меню

Бронирования **Столики** Меню Отчёты

Столики

Номер Мест Описание Фото

6 4 У окна, диван + стулья table6.png **Добавить**

Фото	Номер	Мест	Описание	Действие
	3	4	У окна, схема отображается при бронировании	Ред. Удал.

Меню

Название Категория Цена Фото

Паста Горячее 780 pasta.png **Добавить**

Рисунок 8 – Административный шаблон управления столиками и меню

На рисунке 9 представлен шаблон страницы отчётов и аналитики. В интерфейсе предусмотрены показатели общего количества бронирований, бронирований за месяц, загруженности и отменённых заявок, фильтры по периоду, столику и статусу, а также кнопки скачивания CSV и PDF.

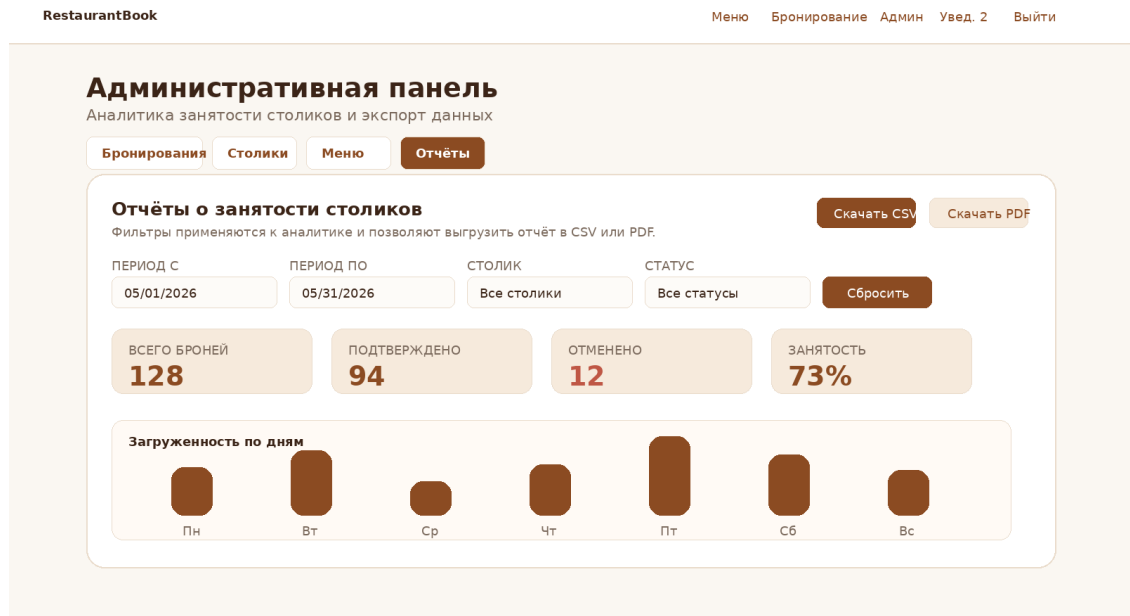


Рисунок 9 – Административный шаблон отчётов и аналитики

Таким образом, во второй главе отражены основные проектные решения: роли пользователей, функциональные требования, варианты использования, структура базы данных, проверка доступности столиков, внутреннее уведомление, email-рассылка, PDF-подтверждения и шаблоны ключевых страниц. Пользовательские шаблоны показывают сценарии регистрации, просмотра меню, бронирования и получения уведомлений, а административные шаблоны — управление столиками, меню, бронированиями и отчётностью.

3 Реализация веб-приложения

3.1 Выбор архитектуры и структуры проекта

Практическая реализация веб-приложения выполнена в соответствии с проектными решениями, приведёнными во второй части курсового проекта. Система построена по клиент-серверной архитектуре: пользователь взаимодействует с интерфейсом React-приложения, клиентская часть отправляет HTTP-запросы к серверу Django REST Framework, а серверная часть выполняет бизнес-логику и сохраняет данные в PostgreSQL.

Выбранная архитектура позволяет разделить ответственность между частями системы: клиентская часть отвечает за отображение интерфейса, обработку форм и маршрутизацию между страницами, серверная часть отвечает за проверку данных, авторизацию, работу с бронированиями, формирование PDF-подтверждений, отправку email-уведомлений и экспорт отчётов. Для воспроизводимого запуска проекта используются Docker и файл `docker-compose.yml`.

Структура реализованного проекта представлена в листинге 1.

```
reservation_service/  
  backend/  
    manage.py  
    requirements.txt  
    reservation_service/  
      settings.py  
      urls.py  
    booking/  
      models.py  
      serializers.py  
      views.py  
      services.py  
      urls.py  
      admin.py  
      tests.py  
  frontend/  
    package.json  
    src/  
      App.jsx  
      api/client.js  
      components/Layout.jsx
```

```
pages/  
styles/style.css  
docker-compose.yml  
.env.example  
paper.tex  
ch1.tex  
ch2.tex  
ch3.tex
```

Листинг 1 – Структура реализованного проекта

3.2 Реализация серверной части

Серверная часть разработана на базе фреймворка Django и библиотеки Django REST Framework. В приложении booking размещены модели данных, сериализаторы, представления API, сервисные функции и настройки административной панели.

Основными сущностями системы являются: пользователь, столик, позиция меню, бронирование, уведомление и файл бронирования. Для пользователя используется стандартная модель User, входящая в Django. Разграничение доступа между обычным авторизованным пользователем и администратором выполняется через поле is_staff. Обычный пользователь может создавать и просматривать только свои бронирования, администратор получает доступ к управлению столиками, меню, бронированиями и отчётами.

Пример модели бронирования представлен в листинге 2.

```
class Booking(models.Model):  
    user = models.ForeignKey(User, on_delete=models.CASCADE)  
    table = models.ForeignKey(RestaurantTable, on_delete=models.PROTECT)  
    date = models.DateField()  
    time_start = models.TimeField()  
    time_end = models.TimeField()  
    guests_count = models.PositiveIntegerField(default=1)  
    status = models.CharField(max_length=20, default='pending')  
    pdf_path = models.CharField(max_length=255, blank=True)
```

Листинг 2 – Фрагмент модели бронирования

Для передачи данных между клиентом и сервером используются сериализаторы. Они преобразуют объекты Django ORM в JSON и выполняют

дополнительную проверку данных. Например, при создании бронирования проверяется корректность временного интервала, соответствие количества гостей вместимости столика и отсутствие пересечений с уже существующими бронированиями.

3.3 Реализация базы данных

В качестве основной базы данных в проекте предусмотрена PostgreSQL. При запуске проекта через Docker создаётся отдельный сервис db, к которому подключается backend. В процессе разработки также предусмотрен резервный вариант с SQLite, если переменные окружения PostgreSQL не заданы. Для развёртывания демонстрационной версии приложения на бесплатном тарифе PythonAnywhere использовалась SQLite, так как она не требует отдельного серверного процесса и подходит для учебной демонстрации функциональности.

В таблице 10 приведены основные таблицы базы данных и их назначение.

Таблица 10 – Основные таблицы базы данных

Таблица	Назначение
auth_user	хранение учётных записей пользователей и администраторов
booking_restauranttable	хранение столиков, вместимости и изображений
booking_menuitem	хранение позиций меню, категорий, цен и изображений блюд
booking_booking	хранение бронирований, даты, времени, статуса и PDF-подтверждения
booking_notification	хранение внутренних и email-уведомлений пользователя
booking_bookingfile	хранение PDF-файлов, связанных с конкретным бронированием

Связь таблиц реализована через внешние ключи. Бронирование связано с пользователем и столиком, уведомление связано с пользователем и, при

необходимости, с конкретным бронированием, а файл подтверждения связан с записью бронирования.

3.4 Реализация аутентификации пользователей

Для входа в систему используется встроенная система пользователей Django и токенная аутентификация Django REST Framework. После регистрации или входа сервер возвращает клиентской части токен. React-приложение сохраняет токен в локальном хранилище и передаёт его в заголовке Authorization при последующих запросах к API.

Фрагмент клиентского кода, отвечающий за добавление токена к запросам, показан в листинге 3.

```
api.interceptors.request.use((config) => {  
  const token = localStorage.getItem('token');  
  if (token) {  
    config.headers.Authorization = 'Token ${token}';  
  }  
  return config;  
});
```

Листинг 3 – Добавление токена авторизации к запросам

На стороне frontend реализованы страницы регистрации и входа. После успешного входа пользователь переходит к просмотру меню и форме бронирования. Если пользователь является администратором, в интерфейсе отображается ссылка на административные разделы.

3.5 Реализация бронирования и проверки доступности столиков

Одной из ключевых функций системы является проверка доступности столика на выбранные дату и время. Проверка выполняется на сервере перед созданием бронирования. Система ищет активные бронирования того же столика на ту же дату и проверяет пересечение временных интервалов. Если пересечение найдено, создание бронирования блокируется.

Фрагмент функции проверки доступности приведён в листинге 4.

```
def is_table_available(table, date, time_start, time_end):  
  query = Booking.objects.filter(  
    table=table,  
    date=date,  
    status__in=['pending', 'confirmed'],
```

```
.filter(time_start__lt=time_end, time_end__gt=time_start)
return not query.exists()
```

Листинг 4 – Проверка доступности столика

На клиентской стороне реализована форма бронирования. Пользователь указывает дату, время начала, время окончания, количество гостей и выбирает столик. Перед отправкой формы можно выполнить отдельную проверку доступности, после которой в интерфейсе отображается сообщение о том, доступен ли столик для выбранного времени.

Для повышения наглядности выбора столика в форме бронирования предусмотрен визуальный предпросмотр. После выбора номера столика справа от формы отображается схема выбранного столика. Если администратор загрузил собственное изображение столика, используется оно; если изображение не загружено, отображается стандартная схема по номеру столика. Такой подход не требует изменения структуры базы данных, так как изображение уже связано с сущностью столика через поле `image`, но делает интерфейс более понятным для пользователя.

3.6 Реализация административной панели

Административная часть реализована на двух уровнях. Первый уровень - стандартная административная панель Django, позволяющая работать с моделями напрямую. Второй уровень - пользовательский административный интерфейс на React, который использует REST API и отображает основные функции, предусмотренные проектом.

В административной части реализованы следующие возможности:

- просмотр списка бронирований;
- подтверждение или отмена бронирования;
- управление столиками;
- управление позициями меню;
- загрузка изображений столиков и блюд;
- формирование отчётов о занятости столиков;
- экспорт отчётов в CSV и PDF.

Для защиты административных методов используется проверка роли пользователя. Операции изменения данных доступны только пользователям

с признаком `is_staff`.

3.7 Реализация уведомлений, PDF-подтверждений и email-рассылки

После создания бронирования система формирует внутреннее уведомление, создаёт PDF-подтверждение и отправляет email на адрес пользователя. Уведомление сохраняется в базе данных и отображается в интерфейсе через страницу уведомлений и значок колокольчика в верхней панели приложения. Email-уведомление содержит основные параметры бронирования: дату, время, номер столика, количество гостей и текущий статус.

Для хранения состояния рассылки в модели уведомления используются поля `delivery_channel`, `email_status` и `sent_at`. Это позволяет отслеживать, было ли уведомление отправлено только внутри сайта или дополнительно продублировано на электронную почту.

Фрагмент создания уведомления показан в листинге 5.

```
notification = Notification.objects.create(
    user=booking.user,
    booking=booking,
    title='Booking created',
    message='Booking details were generated by the system.',
    delivery_channel='site_email',
)
```

Листинг 5 – Создание уведомления о бронировании

В режиме разработки используется консольный backend отправки почты: письма выводятся в лог сервера. Для реальной отправки достаточно указать SMTP-настройки в файле `.env`. Такой подход позволяет не хранить пароль от почтового ящика в коде.

3.8 Реализация клиентской части

Клиентская часть реализована на React. Основной компонент `App.jsx` отвечает за переключение страниц, хранение текущего состояния интерфейса и загрузку количества непрочитанных уведомлений. Запросы к серверу вынесены в отдельный модуль `api/client.js`. Это упрощает поддержку проекта, потому что базовый адрес API и добавление токена авторизации настраиваются в одном месте.

В приложении реализованы страницы:

- вход и регистрация;
- просмотр меню;
- создание бронирования;
- просмотр уведомлений;
- управление бронированиями;
- управление столиками;
- управление меню;
- отчёты и экспорт данных.

Интерфейс оформлен с помощью CSS и адаптирован для просмотра на экранах разной ширины. В верхней панели отображается название приложения, навигация и значок колокольчика с количеством непрочитанных уведомлений.

3.9 Запуск и тестирование приложения

Для запуска проекта используется Docker Compose. Перед запуском необходимо создать файл `.env` на основе `.env.example`. После этого приложение запускается командой, приведённой в листинге 6.

```
docker compose up --build
```

Листинг 6 – Запуск приложения

После запуска доступны следующие адреса:

- frontend: `http://localhost:5173`;
- backend api: `http://localhost:8000/api/`;
- административная панель Django: `http://localhost:8000/admin/`.

Для проверки серверной логики были добавлены автоматические тесты Django REST Framework. Тесты размещены в файле `booking/tests.py` и проверяют наиболее важные функции приложения: регистрацию и вход пользователя, проверку доступности столика, создание бронирования, защиту от пересечения бронирований, создание уведомлений и PDF-подтверждений, права администратора, загрузку изображений столиков и блюд меню, а также экспорт отчётов.

Запуск автоматических тестов выполняется командой, приведённой в листинге 7.

```
docker compose exec backend python manage.py test
```

Листинг 7 – Запуск автоматических тестов backend

Основные тестовые сценарии приведены в таблице 11.

Таблица 11 – Сценарии тестирования приложения

Проверяемая функция	Сценарий	Ожидаемый результат
Аутентификация	Регистрация нового пользователя и последующий вход в систему	Сервер возвращает токен авторизации
Проверка доступности	Проверка столика при отсутствии пересечений по времени	Столик доступен
Защита от двойного бронирования	Создание бронирования на уже занятый временной интервал	Сервер возвращает ошибку валидации
Создание бронирования	Авторизованный пользователь создаёт бронирование	Создаются бронирование, уведомление и PDF-файл
Права доступа	Обычный пользователь пытается создать столик	Сервер запрещает операцию
Административное управление	Администратор создаёт столик и загружает изображение	Столик сохраняется с изображением
Управление меню	Администратор создаёт позицию меню и загружает фото блюда	Блюдо сохраняется с изображением

Проверяемая функция	Сценарий	Ожидаемый результат
Отчёты	Администратор запрашивает CSV и PDF отчёты	Сервер возвращает файлы отчётов

Дополнительно было выполнено ручное тестирование клиентской части. Вручную проверялись отображение страниц, работа формы бронирования, визуальный предпросмотр выбранного столика, отображение уведомлений через значок колокольчика, административные разделы и корректность загрузки изображений через интерфейс администратора. Такой подход позволяет проверить не только серверную логику, но и соответствие интерфейса проектным шаблонам.

В результате тестирования подтверждено, что реализованная система выполняет основные функции, предусмотренные заданием курсового проекта.

3.10 Выводы по разделу

Исходный код разработанного веб-приложения размещён в репозитории GitHub по адресу https://github.com/anastasiaantipovamax/reservation_service. Репозиторий содержит серверную часть на Django REST Framework, клиентскую часть на React, настройки Docker Compose, а также файлы пояснительной записки. Веб-приложение развёрнуто на платформе PythonAnywhere и доступно по адресу <https://anastasiaantipova.pythonanywhere.com/>.

В ходе реализации была создана рабочая версия веб-приложения для автоматизации бронирования столиков в ресторане. Серверная часть на Django REST Framework обеспечивает хранение данных, аутентификацию, проверку доступности столиков, формирование PDF-подтверждений, уведомления и экспорт отчётов. Клиентская часть на React предоставляет интерфейс для авторизованного пользователя и администратора. Основной сценарий запуска с PostgreSQL и Docker делает приложение воспроизводимым и приближает его к реальному сценарию эксплуатации, а демонстрационное размещение на PythonAnywhere с SQLite позволило показать работу проекта на бесплатном хостинге без изменения пользовательской функциональности.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсового проекта была рассмотрена предметная область онлайн-бронирования столиков в ресторане, проведён обзор существующих программных решений и определены требования к разрабатываемому веб-приложению RestaurantBook.

В проектной части были выделены роли пользователей, описаны пользовательские истории, построена диаграмма вариантов использования, спроектирована модель данных и подготовлены шаблоны ключевых страниц. Особое внимание уделено функциям, которые отличают разрабатываемую систему от типовых решений: проверке доступности столиков, генерации PDF-подтверждений, системе уведомлений и экспорту отчётов в форматах CSV и PDF.

Результатом работы является проект веб-приложения, который может быть использован как основа для дальнейшей реализации и развёртывания в ресторане. Дальнейшее развитие проекта может включать интеграцию с платёжными сервисами, расширенную аналитику посещаемости и адаптацию интерфейса под мобильные устройства.

Разработанное веб-приложение было развёрнуто на платформе PythonAnywhere и доступно по адресу <https://anastasiaantipova.pythonanywhere.com/>. Исходный код проекта размещён в репозитории GitHub и предоставлен преподавателю для проверки. Развёртывание приложения на хостинге позволило проверить работоспособность системы в условиях, приближенных к реальной эксплуатации.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Restoplace: сервис бронирования столиков для ресторанов. — URL: <https://restoplace.cc/> (дата обр. 24.04.2026).
2. Yclients: облачная платформа автоматизации сервиса. — URL: <https://www.yclients.com/> (дата обр. 24.04.2026).
3. iiko: система автоматизации ресторанного бизнеса. — URL: <https://iiko.ru/> (дата обр. 24.04.2026).
4. React Documentation. — URL: <https://react.dev/> (дата обр. 24.04.2026).
5. Django Documentation. — URL: <https://docs.djangoproject.com/> (дата обр. 24.04.2026).
6. PostgreSQL Documentation. — URL: <https://www.postgresql.org/docs/> (дата обр. 24.04.2026).
7. Docker Documentation. — URL: <https://docs.docker.com/> (дата обр. 24.04.2026).
8. *Куликов С.* Тестирование программного обеспечения. Базовый курс. — Минск: Четыре четверти, 2017.
9. *Орлов С. А.* Технологии разработки программного обеспечения. — Санкт-Петербург: Питер, 2012.
10. *Леоненков А. В.* Самоучитель UML 2. — Санкт-Петербург: БХВ-Петербург, 2007.
11. *Кириллов В. В., Громов Г. Ю.* Введение в реляционные базы данных. — Санкт-Петербург: БХВ-Петербург, 2012.